

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



ANDROID BASIC WITH KOTLIN

Oleh:

ERISCHA MARSELA

NIM. 2410817120022

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: Android Basic With Kotlin Sederhana ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman II. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : ERISCHA MARSELA
NIM : 2410817120022

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI.....	3
DAFTAR TABEL	4
DAFTAR GAMBAR	5
SOAL 1	6
A. SOURCE CODE	6
B. OUTPUT CODE	58
C. PEMBAHASAN	59
TAUTAN GIT.....	63

DAFTAR TABEL

Tabel 1. Source Code MainActivity Compose.....	6
Tabel 2. AnimeApp.kt Compose.....	8
Tabel 3. AnimeListViewModel.kt Compose	9
Tabel 4. NavGraph.kt Compose	11
Tabel 5. SearchScreen.kt Compose.....	14
Tabel 6. MainActivity.kt XML	16
Tabel 7. MyApplication.kt XML	19
Tabel 8, AnimeViewModel.kt XML.....	19
Tabel 9. AnimeViewModelFactory.kt XML.....	22
Tabel 10. AnimeAdapter.kt XML	23
Tabel 11. AnimeItem.kt XML.....	26
Tabel 12. DetailFragment.kt XML.....	27
Tabel 13. activity_main.xml	33
Tabel 14.fragment_detail.xml	35
Tabel 15. item_anime.xml.....	47
Tabel 16.nav_graph.xml.....	52

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose	58
Gambar 2. Screenshot Hasil Jawaban Soal 1 XML	59

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out SOURCE CODE

MainActivity.Kt Compose

Tabel 1. Source Code MainActivity Compose

1.	package com.example.clickanime
2.	
3.	import android.content.Context
4.	import android.os.Bundle
5.	import androidx.activity.ComponentActivity
6.	import androidx.activity.compose.setContent
7.	import androidx.activity.enableEdgeToEdge
8.	import androidx.compose.foundation.layout.fillMaxSize
9.	import androidx.compose.material3.Surface
10.	import androidx.compose.runtime.remember
11.	import androidx.compose.ui.Modifier

12.	import androidx.lifecycle.viewmodel.compose.viewModel
13.	import androidx.navigation.compose.rememberNavController
14.	import com.example.clickanime.navigation.NavGraph
15.	import com.example.clickanime.ui.theme.AnimeAppTheme
16.	import com.example.clickanime.viewmodel.AnimeListViewModel
17.	import com.example.clickanime.viewmodel.AnimeListViewModelFactory
18.	import java.util.Locale
19.	
20.	class MainActivity : ComponentActivity() {
21.	
22.	override fun attachBaseContext(newBase: Context) {
23.	val prefs = newBase.getSharedPreferences("settings", Context.MODE_PRIVATE)
24.	val lang = prefs.getString("language", Locale.getDefault().language) ?: "en"
25.	val locale = Locale(lang)
26.	Locale.setDefault(locale)
27.	val config = newBase.resources.configuration
28.	config.setLocale(locale)
29.	super.attachBaseContext(newBase.createConfigurationContext(config))
30.	}
31.	
32.	override fun onCreate(savedInstanceState: Bundle?) {
33.	super.onCreate(savedInstanceState)
34.	enableEdgeToEdge()
35.	setContent {
36.	AnimeAppTheme {
37.	Surface(modifier = Modifier.fillMaxSize()) {

38.	val navController = rememberNavController()
39.	val factory = remember {
	AnimeListViewModelFactory("AnimeList") }
40.	val animeListViewModel: AnimeListViewModel =
	viewModel(factory = factory)
41.	NavGraph(
42.	navController = navController,
43.	viewModel = animeListViewModel,
44.	onLocaleChange = { langCode -> applyLocale(langCode)
	}
45.	)
46.	}
47.	}
48.	}
49.	}
50.	
51.	private fun applyLocale(langCode: String) {
52.	getSharedPreferences("settings", Context.MODE_PRIVATE)
53.	.edit()
54.	.putString("language", langCode)
55.	.apply()
56.	recreate()
57.	}
58.	}

Tabel 2. AnimeApp.kt Compose

1.	package com.example.clickanime
2.	
3.	import android.app.Application
4.	import timber.log.Timber
5.	

6.	class AnimeApp : Application() {
7.	override fun onCreate() {
8.	super.onCreate()
9.	if (BuildConfig.DEBUG) {
10.	Timber.plant(Timber.DebugTree())
11.	}
12.	}
13.	}

Tabel 3. AnimeListViewModel.kt Compose

1.	package com.example.clickanime.viewmodel
2.	
3.	import androidx.lifecycle.ViewModel
4.	import androidx.lifecycle.ViewModelProvider
5.	import com.example.clickanime.data.AnimeData
6.	import com.example.clickanime.model.AnimeCharacter
7.	import kotlinx.coroutines.flow.MutableStateFlow
8.	import kotlinx.coroutines.flow.StateFlow
9.	import kotlinx.coroutines.flow.asStateFlow
10.	import timber.log.Timber
11.	
12.	class AnimeListViewModel(private val logTag: String) : ViewModel() {
13.	
14.	private val _characters = MutableStateFlow<List<AnimeCharacter>>>(emptyList())
15.	val characters: StateFlow<List<AnimeCharacter>>> = _characters.asStateFlow()
16.	
17.	
18.	private val _selectedCharacterId = MutableStateFlow<Int?>(null)

19.	val selectedCharacterId: StateFlow<Int?> =
20.	_selectedCharacterId.asStateFlow()
21.	
22.	private val _openMalUrl = MutableStateFlow<String?>(null)
23.	val openMalUrl: StateFlow<String?> = _openMalUrl.asStateFlow()
24.	
25.	init {
26.	val list = AnimeData.characters
27.	_characters.value = list
28.	Timber.tag(logTag).i("Loaded %d characters into list", list.size)
29.	}
30.	
31.	fun onCharacterClicked(id: Int) {
32.	_selectedCharacterId.value = id
33.	val character = _characters.value.find { it.id == id }
34.	character?.let {
35.	Timber.tag(logTag).i(
36.	"Character selected: id=%d nameResId=%d animeResId=%d",
37.	it.id,
38.	it.nameResId,
39.	it.animeResId
40.	)
41.	} ?: Timber.tag(logTag).w("Character clicked but not found: id=%d", id)
42.	}
43.	
44.	fun clearSelectedCharacter() {
45.	_selectedCharacterId.value = null
46.	}

47.	
48.	fun onOpenMalClicked(url: String) {
49.	_openMalUrl.value = url
50.	Timber.tag(logTag).i("Open MAL pressed for url=%s", url)
51.	}
52.	
53.	fun clearOpenMalEvent() {
54.	_openMalUrl.value = null
55.	}
56.	}
57.	
58.	class AnimeListViewModelFactory(private val logTag: String) : ViewModelProvider.Factory {
59.	override fun <T : ViewModel> create(modelClass: Class<T>): T {
60.	if (modelClass.isAssignableFrom(AnimeListViewModel::class.java)) {
61.	@Suppress("UNCHECKED_CAST")
62.	return AnimeListViewModel(logTag) as T
63.	}
64.	throw IllegalArgumentException("Unknown ViewModel class")
65.	}
66.	}

Tabel 4. NavGraph.kt Compose

1.	package com.example.clickanime.navigation
2.	
3.	import androidx.compose.runtime.Composable
4.	import androidx.navigation.NavHostController
5.	import androidx.navigation.NavType
6.	import androidx.navigation.compose.NavHost

7.	import androidx.navigation.compose.composable
8.	import androidx.navigation.navArgument
9.	import com.example.clickanime.ui.detail.DetailScreen
10.	import com.example.clickanime.ui.home.HomeScreen
11.	import com.example.clickanime.ui.language.LanguageScreen
12.	import com.example.clickanime.ui.favorites.FavoritesScreen
13.	import com.example.clickanime.ui.search.SearchScreen
14.	import com.example.clickanime.ui.settings.SettingsScreen
15.	import com.example.clickanime.viewmodel.AnimeListViewModel
16.	
17.	@Composable
18.	fun NavGraph(
19.	navController: NavHostController,
20.	viewModel: AnimeListViewModel,
21.	onLocaleChange: (String) -> Unit
22.	) {
23.	NavHost(
24.	navController = navController,
25.	startDestination = Screen.Home.route
26.	) {
27.	composable(Screen.Home.route) {
28.	HomeScreen(
29.	viewModel = viewModel,
30.	onDetailNavigate = { characterId ->
31.	viewModel.onCharacterClicked(characterId)
32.	navController.navigate(Screen.Detail.createRoute(characterId))
33.	},
34.	onLanguageClick = { navController.navigate(Screen.Language.route) },

35.	onFavoritesClick	=	{
	navController.navigate(Screen.Favorites.route) },		
36.	onSearchClick	=	{
	navController.navigate(Screen.Search.route) },		
37.	onSettingsClick	=	{
	navController.navigate(Screen.Settings.route) }		
38.	)		
39.	}		
40.			
41.	composable(		
42.	route = Screen.Detail.route,		
43.	arguments = listOf(navArgument("characterId") { type =		
	NavType.IntType })		
44.	) { backStackEntry ->		
45.	val characterId	=	
	backStackEntry.arguments?.getInt("characterId") ?: return@composable		
46.	val character = viewModel.characters.value.find { it.id ==		
	characterId }		
47.	character?.let {		
48.	DetailScreen(		
49.	character = it,		
50.	onBack = { navController.popBackStack() },		
51.	onOpenMal = { url ->		
52.	viewModel.onOpenMalClicked(url)		
53.	}		
54.	)		
55.	}		
56.	}		
57.			
58.	composable(Screen.Language.route) {		
59.	LanguageScreen(		

60.	onLanguageSelected = { langCode ->
61.	onLocaleChange(langCode)
62.	navController.popBackStack()
63.	},
64.	onBack = { navController.popBackStack() }
65.	)
66.	}
67.	
68.	composable(Screen.Favorites.route) {
69.	FavoritesScreen(onBack = { navController.popBackStack() })
70.	}
71.	
72.	composable(Screen.Search.route) {
73.	SearchScreen(onBack = { navController.popBackStack() },
	viewModel = viewModel)
74.	}
75.	
76.	composable(Screen.Settings.route) {
77.	SettingsScreen(onBack = { navController.popBackStack() })
78.	}
79.	
80.	
81.	}
82.	}

Tabel 5. SearchScreen.kt Compose

1.	package com.example.clickanime.ui.search
2.	
3.	import androidx.compose.foundation.layout.*

4.	import androidx.compose.material.icons.Icons	
5.	import androidx.compose.material.icons.filled.ArrowBack	
6.	import androidx.compose.material3.*	
7.	import androidx.compose.runtime.*	
8.	import androidx.compose.ui.Modifier	
9.	import androidx.compose.ui.unit.dp	
10.	import com.example.clickanime.viewmodel.AnimeListViewModel	
11.		
12.	@OptIn(ExperimentalMaterial3Api::class)	
13.	@Composable	
14.	fun SearchScreen(	
15.	onBack: () -> Unit,	
16.	viewModel: AnimeListViewModel	
17.	) {	
18.	val characters by viewModel.characters.collectAsState()	
19.	var query by remember { mutableStateOf("") }	
20.	val results = remember(query, characters) {	
21.	if (query.isBlank()) characters else characters.filter {	
22.	it.year.toString().contains(query)	
	it.rating.toString().contains(query)	
23.	}	
24.	}	
25.		
26.	Scaffold(	
27.	topBar = {	
28.	TopAppBar(	
29.	title = { Text("Search") },	
30.	navigationIcon = {	
31.	IconButton(onClick = onBack) {	

32.	Icon(imageVector = Icons.Default.ArrowBack, contentDescription = "Back")
33.	}
34.	}
35.	)
36.	}
37.	) { paddingValues ->
38.	Column(modifier = Modifier.fillMaxSize().padding(paddingValues).padding(16.dp)) {
39.	OutlinedTextField(value = query, onChange = { query = it , label = { Text("Search by year or rating (demo)") }, modifier = Modifier.fillMaxWidth())
40.	Spacer(modifier = Modifier.height(12.dp))
41.	Text(text = "Results: \${results.size}", style = MaterialTheme.typography.bodyMedium)
42.	Spacer(modifier = Modifier.height(8.dp))
43.	results.forEach { item ->
44.	Text(text = "\${item.id} • \${item.year} • \${item.rating}", modifier = Modifier.padding(vertical = 6.dp))
45.	Divider()
46.	}
47.	}
48.	}
49.	}

Tabel 6. MainActivity.kt XML

1.	package com.example.animepopular
2.	
3.	import android.content.Context
4.	import android.os.Bundle
5.	import android.view.View

6.	import androidx.appcompat.app.AppCompatActivity
7.	import androidx.navigation.fragment.NavHostFragment
8.	import androidx.navigation.ui.AppBarConfiguration
9.	import androidx.navigation.ui.setupActionBarWithNavController
10.	import androidx.navigation.ui.setupWithNavController
11.	import com.example.animepopular.databinding.ActivityMainBinding
12.	import java.util.Locale
13.	
14.	class MainActivity : AppCompatActivity() {
15.	
16.	private lateinit var binding: ActivityMainBinding
17.	
18.	override fun attachBaseContext(newBase: Context) {
19.	val prefs = newBase.getSharedPreferences("settings", Context.MODE_PRIVATE)
20.	val lang = prefs.getString("language", "en") ?: "en"
21.	val locale = Locale(lang)
22.	Locale.setDefault(locale)
23.	val config = newBase.resources.configuration
24.	config.setLocale(locale)
25.	val context = newBase.createConfigurationContext(config)
26.	super.attachBaseContext(context)
27.	}
28.	
29.	override fun onCreate(savedInstanceState: Bundle?) {
30.	super.onCreate(savedInstanceState)
31.	binding = ActivityMainBinding.inflate(layoutInflater)
32.	setContentView(binding.root)
33.	

34.	val navHostFragment = supportFragmentManager	
35.	.findFragmentById(R.id.nav_host_fragment)	as
	NavHostFragment	
36.	val navController = navHostFragment.navController	
37.		
38.	setSupportActionBar(binding.toolbar)	
39.		
40.	val topLevelDestinations = setOf(	
41.	R.id.homeFragment,	
42.	R.id.searchFragment,	
43.	R.id.favoritesFragment,	
44.	R.id.topRatedFragment,	
45.	R.id.profileFragment	
46.	)	
47.		
48.	val appBarConfig = AppBarConfiguration(topLevelDestinations)	
49.	setupActionBarWithNavController(navController, appBarConfig)	
50.		
51.	binding.bottomNavigationView.setupWithNavController(navController)	
52.		
53.	navController.addOnDestinationChangeListener { _, destination, _	
	->	
54.	if (destination.id in topLevelDestinations) {	
55.	binding.bottomNavigationView.visibility = View.VISIBLE	
56.	} else {	
57.	binding.bottomNavigationView.visibility = View.GONE	
58.	}	
59.	supportActionBar?.title = destination.label	
60.	}	
61.	}	

62.	
63.	override fun onSupportNavigateUp(): Boolean {
64.	val navHostFragment = supportFragmentManager
65.	.findFragmentById(R.id.nav_host_fragment) as NavHostFragment
66.	return navHostFragment.navController.navigateUp() super.onSupportNavigateUp()
67.	}
68.	}

Tabel 7. MyApplication.kt XML

1.	package com.example.animepopular
2.	
3.	import android.app.Application
4.	import timber.log.Timber
5.	
6.	class MyApplication : Application() {
7.	override fun onCreate() {
8.	super.onCreate()
9.	
10.	if (BuildConfig.DEBUG) {
11.	Timber.plant(Timber.DebugTree())
12.	Timber.d("Application: Timber logging initialized")
13.	}
14.	}
15.	}

Tabel 8, AnimeViewModel.kt XML

1.	package com.example.animepopular.viewmodel
2.	

3.	import androidx.lifecycle.ViewModel
4.	import com.example.animepopular.data.AnimeRepository
5.	import com.example.animepopular.model.AnimeItem
6.	import kotlinx.coroutines.flow.MutableStateFlow
7.	import kotlinx.coroutines.flow.StateFlow
8.	import timber.log.Timber
9.	
10.	class AnimeViewModel(private val filterParam: String = "all") : ViewModel() {
11.	
12.	private val _animeList = MutableStateFlow<List<AnimeItem>>(emptyList())
13.	val animeList: StateFlow<List<AnimeItem>> = _animeList
14.	
15.	private val _selectedAnime = MutableStateFlow<AnimeItem?>(null)
16.	val selectedAnime: StateFlow<AnimeItem?> = _selectedAnime
17.	
18.	private val _detailButtonClicked = MutableStateFlow(false)
19.	val detailButtonClicked: StateFlow<Boolean> = _detailButtonClicked
20.	
21.	private val _explicitIntentClicked = MutableStateFlow(false)
22.	val explicitIntentClicked: StateFlow<Boolean> = _explicitIntentClicked
23.	
24.	init {
25.	loadAnimeList()
26.	}
27.	
28.	private fun loadAnimeList() {
29.	val list = AnimeRepository.getAnimeList()

30.	<code>_animeList.value = list</code>
31.	<code>Timber.d("Anime list loaded with \${list.size} items")</code>
32.	<code>}</code>
33.	
34.	<code>fun onDetailButtonClicked(anime: AnimeItem) {</code>
35.	<code>Timber.d("Detail button clicked for anime: \${anime.titleEn} (ID: \${anime.id})")</code>
36.	<code>_selectedAnime.value = anime</code>
37.	<code>_detailButtonClicked.value = true</code>
38.	<code>Timber.d("Selected anime data - Title: \${anime.titleEn}, Year: \${anime.year}, Rating: \${anime.rating}")</code>
39.	<code>}</code>
40.	
41.	<code>fun onExplicitIntentClicked(anime: AnimeItem) {</code>
42.	<code>Timber.d("Explicit Intent button clicked for anime: \${anime.titleEn} (ID: \${anime.id})")</code>
43.	<code>_selectedAnime.value = anime</code>
44.	<code>_explicitIntentClicked.value = true</code>
45.	<code>}</code>
46.	
47.	<code>fun resetDetailButtonState() {</code>
48.	<code>_detailButtonClicked.value = false</code>
49.	<code>}</code>
50.	
51.	<code>fun resetExplicitIntentState() {</code>
52.	<code>_explicitIntentClicked.value = false</code>
53.	<code>}</code>
54.	
55.	<code>fun addAnimeToList(anime: AnimeItem) {</code>

56.	Timber.d("Adding anime to list: \${anime.titleEn} (ID: \${anime.id})")
57.	val currentList = _animeList.value.toMutableList()
58.	if (!currentList.any { it.id == anime.id }) {
59.	currentList.add(anime)
60.	_animeList.value = currentList
61.	Timber.d("Anime added successfully. Total items: \${currentList.size}")
62.	} else {
63.	Timber.w("Anime already exists in list: \${anime.titleEn}")
64.	}
65.	}
66.	
67.	fun getFilterParam(): String {
68.	Timber.d("Filter parameter: \$filterParam")
69.	return filterParam
70.	}
71.	}

Tabel 9. AnimeViewModelFactory.kt XML

1.	package com.example.animepopular.viewmodel
2.	
3.	import androidx.lifecycle.ViewModel
4.	import androidx.lifecycle.ViewModelProvider
5.	
6.	class AnimeViewModelFactory(private val filterParam: String = "all") : ViewModelProvider.Factory {
7.	
8.	override fun <T : ViewModel> create(modelClass: Class<T>): T {
9.	if (modelClass.isAssignableFrom(AnimeViewModel::class.java)) {

10.	<code>@Suppress("UNCHECKED_CAST")</code>
11.	<code>return AnimeViewModel(filterParam) as T</code>
12.	<code>}</code>
13.	<code>throw IllegalArgumentException("Unknown ViewModel class")</code>
14.	<code>}</code>
15.	<code>}</code>

Tabel 10. AnimeAdapter.kt XML

1.	<code>package com.example.animepopular.adapter</code>
2.	
3.	<code>import android.content.Context</code>
4.	<code>import android.view.LayoutInflater</code>
5.	<code>import android.view.ViewGroup</code>
6.	<code>import androidx.core.content.ContextCompat</code>
7.	<code>import androidx.recyclerview.widget.DiffUtil</code>
8.	<code>import androidx.recyclerview.widget.ListAdapter</code>
9.	<code>import androidx.recyclerview.widget.RecyclerView</code>
10.	<code>import com.bumptech.glide.Glide</code>
11.	<code>import com.example.animepopular.R</code>
12.	<code>import com.example.animepopular.data.FavoritesRepository</code>
13.	<code>import com.example.animepopular.databinding.ItemAnimeBinding</code>
14.	<code>import com.example.animepopular.model.AnimeItem</code>
15.	
16.	<code>class AnimeAdapter(</code>
17.	<code>private val onImdbClick: (AnimeItem) -> Unit,</code>
18.	<code>private val onDetailClick: (AnimeItem) -> Unit,</code>
19.	<code>private val onFavoriteToggled: ((AnimeItem, isFavorite: Boolean) -> Unit)? = null</code>
20.	<code>) : ListAdapter<AnimeItem, AnimeAdapter.AnimeViewHolder>(DiffCallback()) {</code>

21.	
22.	override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): AnimeViewHolder {
23.	val binding = ItemAnimeBinding.inflate(
24.	LayoutInflater.from(parent.context), parent, false
25.	)
26.	return AnimeViewHolder(binding, parent.context)
27.	}
28.	
29.	override fun onBindViewHolder(holder: AnimeViewHolder, position: Int) {
30.	holder.bind(getItem(position))
31.	}
32.	
33.	inner class AnimeViewHolder(
34.	private val binding: ItemAnimeBinding,
35.	private val context: Context
36.	): RecyclerView.ViewHolder(binding.root) {
37.	
38.	fun bind(anime: AnimeItem) {
39.	val prefs = context.getSharedPreferences("settings", Context.MODE_PRIVATE)
40.	val isId = prefs.getString("language", "en") == "id"
41.	
42.	binding.tvTitle.text = if (isId) anime.titleId else anime.titleEn
43.	binding.tvYear.text = anime.year.toString()
44.	binding.tvGenre.text = if (isId) anime.genreId else anime.genreEn
45.	binding.tvPlot.text = if (isId) anime.plotId else anime.plotEn
46.	binding.tvRating.text = anime.rating

47.	binding.tvEpisodes.text = anime.episodes
48.	
49.	Glide.with(context)
50.	.load(anime.imageUrl)
51.	.placeholder(android.R.drawable.ic_menu_gallery)
52.	.error(android.R.drawable.ic_menu_gallery)
53.	.into(binding.imgPoster)
54.	
55.	binding.btnImdb.setOnClickListener { onImdbClick(anime) }
56.	binding.btnDetail.setOnClickListener { onDetailClick(anime) }
57.	
58.	updateFavoriteIcon(anime)
59.	
60.	binding.btnFavorite.setOnClickListener {
61.	FavoritesRepository.toggleFavorite(anime.id)
62.	updateFavoriteIcon(anime)
63.	onFavoriteToggled?.invoke(anime,
	FavoritesRepository.isFavorite(anime.id))
64.	}
65.	}
66.	
67.	private fun updateFavoriteIcon(anime: AnimeItem) {
68.	val isFav = FavoritesRepository.isFavorite(anime.id)
69.	if (isFav) {
70.	binding.btnFavorite.setIconResource(R.drawable.ic_favorite)
71.	binding.btnFavorite.iconTint =
72.	ContextCompat.getColorStateList(context,
	R.color.rating_color)
73.	binding.btnFavorite.strokeColor =

74.	ContextCompat.getColorStateList(context, R.color.rating_color)
75.	} else {
76.	binding.btnFavorite.setIconResource(R.drawable.ic_favorite_border)
77.	binding.btnFavorite.iconTint =
78.	ContextCompat.getColorStateList(context, R.color.text_secondary)
79.	binding.btnFavorite.strokeColor =
80.	ContextCompat.getColorStateList(context, R.color.text_secondary)
81.	}
82.	}
83.	}
84.	
85.	class DiffCallback : DiffUtil.ItemCallback<AnimeItem>() {
86.	override fun areItemsTheSame(oldItem: AnimeItem, newItem: AnimeItem) =
87.	oldItem.id == newItem.id
88.	override fun areContentsTheSame(oldItem: AnimeItem, newItem: AnimeItem) =
89.	oldItem == newItem
90.	}
91.	}

Tabel 11. AnimeItem.kt XML

1.	package com.example.animepopular.model
2.	
3.	import android.os.Parcelable
4.	import kotlinx.parcelize.Parcelize
5.	

6.	@Parcelize
7.	data class AnimeItem(
8.	val id: Int,
9.	val titleEn: String,
10.	val titleId: String,
11.	val year: Int,
12.	val genreEn: String,
13.	val genreId: String,
14.	val plotEn: String,
15.	val plotId: String,
16.	val studioEn: String,
17.	val studioId: String,
18.	val episodes: String,
19.	val rating: String,
20.	val imageUrl: String,
21.	val imdbUrl: String
22.	) : Parcelable

Tabel 12. DetailFragment.kt XML

1.	package com.example.animepopular.ui.detail
2.	
3.	import android.content.Context
4.	import android.content.Intent
5.	import android.net.Uri
6.	import android.os.Bundle
7.	import android.view.LayoutInflater
8.	import android.view.View
9.	import android.view.ViewGroup
10.	import android.widget.Toast

11.	import androidx.core.content.ContextCompat
12.	import androidx.fragment.app.Fragment
13.	import androidx.navigation.fragment.navArgs
14.	import androidx.recyclerview.widget.LinearLayoutManager
15.	import com.bumptech.glide.Glide
16.	import com.example.animepopular.R
17.	import com.example.animepopular.adapter.ReviewAdapter
18.	import com.example.animepopular.data.FavoritesRepository
19.	import com.example.animepopular.databinding.FragmentDetailBinding
20.	import com.example.animepopular.model.AnimeItem
21.	import com.example.animepopular.model.Review
22.	import java.text.SimpleDateFormat
23.	import java.util.Date
24.	import java.util.Locale
25.	
26.	class DetailFragment : Fragment() {
27.	
28.	private var _binding: FragmentDetailBinding? = null
29.	private val binding get() = _binding!!
30.	
31.	private val args: DetailFragmentArgs by navArgs()
32.	
33.	private lateinit var reviewAdapter: ReviewAdapter
34.	
35.	override fun onCreateView(
36.	inflater: LayoutInflater, container: ViewGroup?,
37.	savedInstanceState: Bundle?
38.	): View {
39.	_binding = FragmentDetailBinding.inflate(inflater, container, false)

40.	return binding.root
41.	}
42.	
43.	override fun onCreateView(view: View, savedInstanceState: Bundle?) {
44.	super.onCreateView(view, savedInstanceState)
45.	
46.	val anime = args.anime
47.	val isIndonesian = getCurrentLanguage() == "id"
48.	
49.	bindData(anime, isIndonesian)
50.	setupWatchedButton(anime)
51.	setupReviewSection(anime)
52.	}
53.	
54.	private fun bindData(anime: AnimeItem, isIndonesian: Boolean) {
55.	Glide.with(this)
56.	.load(anime.imageUrl)
57.	.placeholder(android.R.drawable.ic_menu_gallery)
58.	.error(android.R.drawable.ic_menu_gallery)
59.	.centerCrop()
60.	.into(binding.imgDetailPoster)
61.	
62.	binding.tvDetailTitle.text = if (isIndonesian) anime.titleId else anime.titleEn
63.	binding.tvDetailYear.text = anime.year.toString()
64.	binding.tvDetailGenre.text = if (isIndonesian) anime.genreId else anime.genreEn
65.	binding.tvDetailStudio.text = if (isIndonesian) anime.studioId else anime.studioEn

66.	binding.tvDetailEpisodes.text = anime.episodes
67.	binding.tvDetailRating.text = anime.rating
68.	binding.tvDetailPlot.text = if (isIndonesian) anime.plotId else anime.plotEn
69.	
70.	binding.btnDetailImdb.setOnClickListener {
71.	startActivity(Intent(Intent.ACTION_VIEW,
72.	Uri.parse(anime.imdbUrl)))
73.	}
74.	}
75.	private fun setupWatchedButton(anime: AnimeItem) {
76.	updateWatchedButtonState(anime.id)
77.	
78.	binding.btnWatched.setOnClickListener {
79.	FavoritesRepository.toggleWatched(anime.id)
80.	updateWatchedButtonState(anime.id)
81.	}
82.	}
83.	
84.	private fun updateWatchedButtonState(animeId: Int) {
85.	val isWatched = FavoritesRepository.isWatched(animeId)
86.	if (isWatched) {
87.	binding.btnWatched.setIconResource(R.drawable.ic_watched)
88.	binding.btnWatched.iconTint =
89.	ContextCompat.getColorStateList(requireContext(),
90.	R.color.accent_color)
91.	binding.btnWatched.strokeColor =
	ContextCompat.getColorStateList(requireContext(),
	R.color.accent_color)

92.	} else {	
93.	binding.btnWatched.setIconResource(R.drawable.ic_watched_border)	
94.	binding.btnWatched.iconTint =	
95.	ContextCompat.getColorStateList(requireContext(),	
	R.color.text_secondary)	
96.	binding.btnWatched.strokeColor =	
97.	ContextCompat.getColorStateList(requireContext(),	
	R.color.text_secondary)	
98.	}	
99.	}	
100.		
101.	private fun setupReviewSection(anime: AnimeItem) {	
102.	reviewAdapter = ReviewAdapter()	
103.	binding.rvReviews.layoutManager	=
	LinearLayoutManager(requireContext())	
104.	binding.rvReviews.adapter = reviewAdapter	
105.	binding.rvReviews.isNestedScrollingEnabled = false	
106.		
107.	refreshReviews(anime.id)	
108.		
109.	binding.btnSendReview.setOnClickListener {	
110.	submitReview(anime.id)	
111.	}	
112.	}	
113.		
114.	private fun submitReview(animeId: Int) {	
115.	val username = binding.etReviewUsername.text.toString().trim()	
116.	val reviewText = binding.etReviewText.text.toString().trim()	
117.		

118.	if (username.isEmpty()) {
119.	binding.etReviewUsername.error = "X"
120.	binding.etReviewUsername.requestFocus()
121.	return
122.	}
123.	if (reviewText.isEmpty()) {
124.	binding.etReviewText.error = "X"
125.	binding.etReviewText.requestFocus()
126.	return
127.	}
128.	
129.	val timestamp = SimpleDateFormat("dd MMM yyyy, HH:mm", Locale.getDefault())
130.	.format(Date())
131.	val review = Review(132. username = username, 133. text = reviewText, 134. timestamp = timestamp 135.)
136.	
137.	FavoritesRepository.addReview(animeId, review)
138.	refreshReviews(animeId)
139.	
140.	binding.etReviewUsername.text?.clear()
141.	binding.etReviewText.text?.clear()
142.	binding.etReviewUsername.clearFocus()
143.	binding.etReviewText.clearFocus()
144.	}
145.	

146.	private fun refreshReviews(animeId: Int) {
147.	val reviews = FavoritesRepository.getReviews(animeId)
148.	reviewAdapter.submitList(reviews)
149.	
150.	binding.tvReviewCount.text = when (reviews.size) {
151.	1 -> "1 review"
152.	else -> "\${reviews.size} review"
153.	}
154.	}
155.	
156.	private fun getCurrentLanguage(): String {
157.	val prefs = requireContext().getSharedPreferences("settings",
	Context.MODE_PRIVATE)
158.	return prefs.getString("language", "en") ?: "en"
159.	}
160.	
161.	override fun onDestroyView() {
162.	super.onDestroyView()
163.	_binding = null
164.	}
165.	}

Tabel 13. activity_main.xml

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<androidx.coordinatorlayout.widget.CoordinatorLayout
3.	xmlns:android="http://schemas.android.com/apk/res/android"
4.	xmlns:app="http://schemas.android.com/apk/res-auto"
5.	android:layout_width="match_parent"
6.	android:layout_height="match_parent"

7.	android:fitsSystemWindows="true">
8.	
9.	<com.google.android.material.appbar.AppBarLayout
10.	android:id="@+id/appBarLayout"
11.	android:layout_width="match_parent"
12.	android:layout_height="wrap_content"
13.	android:theme="@style/ThemeOverlay.MaterialComponents.Dark.ActionBar"
14.	android:fitsSystemWindows="true">
15.	
16.	<androidx.appcompat.widget.Toolbar
17.	android:id="@+id/toolbar"
18.	android:layout_width="match_parent"
19.	android:layout_height="?attr/actionBarSize"
20.	android:background="?attr/colorPrimary"
21.	app:popupTheme="@style/ThemeOverlay.MaterialComponents.Light"
22.	app:title="@string/app_name"/>
23.	
24.	</com.google.android.material.appbar.AppBarLayout>
25.	
26.	<androidx.fragment.app.FragmentContainerView
27.	android:id="@+id/nav_host_fragment"
28.	android:name="androidx.navigation.fragment.NavHostFragment"
29.	android:layout_width="match_parent"
30.	android:layout_height="match_parent"
31.	app:layout_behavior="@string/appbar_scrolling_view_behavior"
32.	app:defaultNavHost="true"
33.	app:navGraph="@navigation/nav_graph"/>
34.	

35.	<com.google.android.material.bottomnavigation.BottomNavigationView
36.	android:id="@+id/bottomNavView"
37.	android:layout_width="match_parent"
38.	android:layout_height="wrap_content"
39.	android:layout_gravity="bottom"
40.	android:background="@color/colorPrimary"
41.	android:fitsSystemWindows="true"
42.	app:itemIconTint="@color/bottom_nav_selector"
43.	app:itemTextColor="@color/bottom_nav_selector"
44.	app:labelVisibilityMode="labeled"
45.	app:menu="@menu/bottom_nav_menu"/>
46.	
47.	</androidx.coordinatorlayout.widget.CoordinatorLayout>

Tabel 14.fragment_detail.xml

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<androidx.core.widget.NestedScrollView
3.	xmlns:android="http://schemas.android.com/apk/res/android"
4.	xmlns:app="http://schemas.android.com/apk/res-auto"
5.	android:layout_width="match_parent"
6.	android:layout_height="match_parent"
7.	android:background="@color/background_dark"
8.	android:clipToPadding="false">
9.	
10.	<LinearLayout
11.	android:layout_width="match_parent"
12.	android:layout_height="wrap_content"
13.	android:orientation="vertical">

14.	
15.	<ImageView
16.	android:id="@+id/imgDetailPoster"
17.	android:layout_width="match_parent"
18.	android:layout_height="0dp"
19.	android:layout_weight="1"
20.	android:minHeight="160dp"
21.	android:maxHeight="260dp"
22.	android:scaleType="centerCrop"
23.	android:adjustViewBounds="false"
24.	android:src="@drawable/ic_placeholder"/>
25.	
26.	<View
27.	android:layout_width="match_parent"
28.	android:layout_height="56dp"
29.	android:layout_marginTop="-56dp"
30.	android:background="@drawable/gradient_bottom_fade"/>
31.	
32.	<com.google.android.material.card.MaterialCardView
33.	android:layout_width="match_parent"
34.	android:layout_height="wrap_content"
35.	android:layout_marginHorizontal="12dp"
36.	android:layout_marginTop="-20dp"
37.	android:layout_marginBottom="12dp"
38.	app:cardBackgroundColor="@color/card_background"
39.	app:cardCornerRadius="20dp"
40.	app:cardElevation="6dp">
41.	
42.	<LinearLayout

43.	android:layout_width="match_parent"
44.	android:layout_height="wrap_content"
45.	android:orientation="vertical"
46.	android:padding="16dp">
47.	
48.	<LinearLayout
49.	android:layout_width="match_parent"
50.	android:layout_height="wrap_content"
51.	android:orientation="horizontal"
52.	android:gravity="center_vertical">
53.	
54.	<TextView
55.	android:id="@+id/tvDetailTitle"
56.	android:layout_width="0dp"
57.	android:layout_height="wrap_content"
58.	android:layout_weight="1"
59.	android:textColor="@color/text_primary"
60.	android:textSize="20sp"
61.	android:textStyle="bold"
62.	android:layout_marginEnd="8dp"/>
63.	
64.	<TextView
65.	android:id="@+id/tvDetailYear"
66.	android:layout_width="wrap_content"
67.	android:layout_height="wrap_content"
68.	android:background="@drawable/badge_background"
69.	android:paddingHorizontal="10dp"
70.	android:paddingVertical="4dp"
71.	android:textColor="@color/accent_color"

72.	android:textSize="13sp"
73.	android:textStyle="bold"/>
74.	</LinearLayout>
75.	
76.	<TextView
77.	android:id="@+id/tvDetailGenre"
78.	android:layout_width="wrap_content"
79.	android:layout_height="wrap_content"
80.	android:layout_marginTop="6dp"
81.	android:background="@drawable/badge_background"
82.	android:paddingHorizontal="10dp"
83.	android:paddingVertical="3dp"
84.	android:textColor="@color/accent_color"
85.	android:textSize="12sp"/>
86.	
87.	<View
88.	android:layout_width="match_parent"
89.	android:layout_height="1dp"
90.	android:layout_marginVertical="10dp"
91.	android:background="#3BFFFFFF"/>
92.	
93.	<LinearLayout
94.	android:layout_width="match_parent"
95.	android:layout_height="wrap_content"
96.	android:orientation="horizontal"
97.	android:gravity="center_vertical">
98.	
99.	<TextView
100.	android:layout_width="wrap_content"

101.	android:layout_height="wrap_content"
102.	android:text="@string/label_rating"
103.	android:textColor="@color/text_secondary"
104.	android:textSize="12sp"
105.	android:layout_marginEnd="6dp"/>
106.	
107.	<TextView
108.	android:id="@+id/tvDetailRating"
109.	android:layout_width="0dp"
110.	android:layout_height="wrap_content"
111.	android:layout_weight="1"
112.	android:textColor="@color/rating_color"
113.	android:textSize="15sp"
114.	android:textStyle="bold"/>
115.	
116.	<TextView
117.	android:layout_width="wrap_content"
118.	android:layout_height="wrap_content"
119.	android:text="@string/label_episodes"
120.	android:textColor="@color/text_secondary"
121.	android:textSize="12sp"
122.	android:layout_marginEnd="6dp"/>
123.	
124.	<TextView
125.	android:id="@+id/tvDetailEpisodes"
126.	android:layout_width="wrap_content"
127.	android:layout_height="wrap_content"
128.	android:textColor="@color/text_primary"
129.	android:textSize="12sp"/>

130.	</LinearLayout>
131.	
132.	<LinearLayout
133.	android:layout_width="match_parent"
134.	android:layout_height="wrap_content"
135.	android:orientation="horizontal"
136.	android:gravity="center_vertical"
137.	android:layout_marginTop="6dp">
138.	
139.	<TextView
140.	android:layout_width="wrap_content"
141.	android:layout_height="wrap_content"
142.	android:text="@string/label_studio"
143.	android:textColor="@color/text_secondary"
144.	android:textSize="12sp"
145.	android:layout_marginEnd="6dp"/>
146.	
147.	<TextView
148.	android:id="@+id/tvDetailStudio"
149.	android:layout_width="match_parent"
150.	android:layout_height="wrap_content"
151.	android:textColor="@color/text_primary"
152.	android:textSize="12sp"
153.	android:maxLines="1"
154.	android:ellipsize="end"/>
155.	</LinearLayout>
156.	
157.	<View
158.	android:layout_width="match_parent"

159.	android:layout_height="1dp"
160.	android:layout_marginVertical="10dp"
161.	android:background="#3BFFFFFF"/>
162.	
163.	<TextView
164.	android:layout_width="wrap_content"
165.	android:layout_height="wrap_content"
166.	android:text="@string/label_plot"
167.	android:textColor="@color/text_secondary"
168.	android:textSize="12sp"
169.	android:textStyle="bold"
170.	android:layout_marginBottom="5dp"/>
171.	
172.	<TextView
173.	android:id="@+id/tvDetailPlot"
174.	android:layout_width="match_parent"
175.	android:layout_height="wrap_content"
176.	android:textColor="@color/text_primary"
177.	android:textSize="13sp"
178.	android:lineSpacingMultiplier="1.5"/>
179.	
180.	<LinearLayout
181.	android:layout_width="match_parent"
182.	android:layout_height="wrap_content"
183.	android:orientation="horizontal"
184.	android:layout_marginTop="16dp"
185.	android:gravity="center_vertical">
186.	
187.	<com.google.android.material.button.MaterialButton

188.	android:id="@+id/btnDetailImdb"
189.	android:layout_width="0dp"
190.	android:layout_height="46dp"
191.	android:layout_weight="1"
192.	android:layout_marginEnd="8dp"
193.	android:text="@string/btn_open_imdb"
194.	android:textSize="12sp"
195.	android:paddingHorizontal="8dp"
196.	app:cornerRadius="23dp"
197.	app:backgroundTint="@color/imdb_yellow"
198.	android:textColor="@android:color/black"
199.	app:icon="@drawable/ic_open_external"
200.	app:iconGravity="textStart"
201.	app:iconTint="@android:color/black"/>
202.	
203.	<com.google.android.material.button.MaterialButton
204.	android:id="@+id/btnWatched"
205.	android:layout_width="46dp"
206.	android:layout_height="46dp"
207.	android:padding="0dp"
208.	android:minWidth="0dp"
209.	style="@style/Widget.MaterialComponents.Button.OutlinedButton"
210.	app:cornerRadius="23dp"
211.	app:strokeColor="@color/text_secondary"
212.	app:icon="@drawable/ic_watched_border"
213.	app:iconSize="20dp"
214.	app:iconTint="@color/text_secondary"
215.	app:iconGravity="textStart"

216.	app:iconPadding="0dp"/>
217.	
218.	</LinearLayout>
219.	</LinearLayout>
220.	</com.google.android.material.card.MaterialCardView>
221.	
222.	<com.google.android.material.card.MaterialCardView
223.	android:layout_width="match_parent"
224.	android:layout_height="wrap_content"
225.	android:layout_marginHorizontal="12dp"
226.	android:layout_marginBottom="20dp"
227.	app:cardBackgroundColor="@color/card_background"
228.	app:cardCornerRadius="20dp"
229.	app:cardElevation="6dp">
230.	
231.	<LinearLayout
232.	android:layout_width="match_parent"
233.	android:layout_height="wrap_content"
234.	android:orientation="vertical"
235.	android:padding="16dp">
236.	
237.	<TextView
238.	android:layout_width="wrap_content"
239.	android:layout_height="wrap_content"
240.	android:text="@string/reviewuser"
241.	android:textColor="@color/text_primary"
242.	android:textSize="15sp"
243.	android:textStyle="bold"
244.	android:layout_marginBottom="8dp"/>

245.	
246.	<TextView
247.	android:id="@+id/tvReviewCount"
248.	android:layout_width="wrap_content"
249.	android:layout_height="wrap_content"
250.	android:text="@string/noreview"
251.	android:textColor="@color/text_secondary"
252.	android:textSize="12sp"
253.	android:layout_marginBottom="8dp"/>
254.	
255.	<androidx.recyclerview.widget.RecyclerView
256.	android:id="@+id/rvReviews"
257.	android:layout_width="match_parent"
258.	android:layout_height="wrap_content"
259.	android:nestedScrollingEnabled="false"
260.	android:layout_marginBottom="12dp"/>
261.	
262.	<View
263.	android:layout_width="match_parent"
264.	android:layout_height="1dp"
265.	android:background="#3BFFFFFF"
266.	android:layout_marginBottom="12dp"/>
267.	
268.	<TextView
269.	android:layout_width="wrap_content"
270.	android:layout_height="wrap_content"
271.	android:text="@string/typereview"
272.	android:textColor="@color/text_primary"
273.	android:textSize="13sp"

274.	android:textStyle="bold"
275.	android:layout_marginBottom="10dp"/>
276.	
277.	<com.google.android.material.textfield.TextInputLayout
278.	android:layout_width="match_parent"
279.	android:layout_height="wrap_content"
280.	android:hint="User"
281.	style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
282.	android:layout_marginBottom="10dp"
283.	app:boxStrokeColor="@color/accent_color"
284.	app:hintTextColor="@color/text_secondary">
285.	
286.	<com.google.android.material.textfield.TextInputEditText
287.	android:id="@+id/etReviewUsername"
288.	android:layout_width="match_parent"
289.	android:layout_height="wrap_content"
290.	android:textColor="@color/text_primary"
291.	android:inputType="textPersonName"
292.	android:imeOptions="actionNext"/>
293.	</com.google.android.material.textfield.TextInputLayout>
294.	
295.	<!-- Review Text -->
296.	<com.google.android.material.textfield.TextInputLayout
297.	android:layout_width="match_parent"
298.	android:layout_height="wrap_content"
299.	android:hint="Review"
300.	style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
301.	android:layout_marginBottom="12dp"

302.	app:boxStrokeColor="@color/accent_color"
303.	app:hintTextColor="@color/text_secondary"
304.	app:counterEnabled="true"
305.	app:counterMaxLength="300">
306.	
307.	<com.google.android.material.textfield.TextInputEditText
308.	android:id="@+id/etReviewText"
309.	android:layout_width="match_parent"
310.	android:layout_height="wrap_content"
311.	android:textColor="@color/text_primary"
312.	android:inputType="textMultiLine"
313.	android:minLines="3"
314.	android:maxLines="6"
315.	android:gravity="top start"
316.	android:maxLength="300"/>
317.	</com.google.android.material.textfield.TextInputLayout>
318.	
319.	<!-- Send -->
320.	<com.google.android.material.button.MaterialButton
321.	android:id="@+id/btnSendReview"
322.	android:layout_width="match_parent"
323.	android:layout_height="46dp"
324.	android:text="@string/sendreview"
325.	android:textSize="13sp"
326.	app:cornerRadius="23dp"
327.	app:backgroundTint="@color/accent_color"/>
328.	
329.	</LinearLayout>
330.	</com.google.android.material.card.MaterialCardView>

331.	
332.	</LinearLayout>
333.	
334.	</androidx.core.widget.NestedScrollView>

Tabel 15. item_anime.xml

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<com.google.android.material.card.MaterialCardView
3.	xmlns:android="http://schemas.android.com/apk/res/android"
4.	xmlns:app="http://schemas.android.com/apk/res-auto"
5.	android:layout_width="match_parent"
6.	android:layout_height="wrap_content"
7.	android:layout_marginHorizontal="8dp"
8.	android:layout_marginBottom="12dp"
9.	app:cardBackgroundColor="@color/card_background"
10.	app:cardCornerRadius="16dp"
11.	app:cardElevation="4dp">
12.	
13.	<LinearLayout
14.	android:layout_width="match_parent"
15.	android:layout_height="wrap_content"
16.	android:orientation="horizontal"
17.	android:padding="12dp">
18.	
19.	<com.google.android.material.imageview.ShapeableImageView
20.	android:id="@+id/imgPoster"
21.	android:layout_width="88dp"
22.	android:layout_height="124dp"

23.	android:scaleType="centerCrop"
24.	android:adjustViewBounds="true"
25.	app:shapeAppearanceOverlay="@style/RoundedCornerImage"/>
26.	
27.	
28.	<LinearLayout
29.	android:layout_width="0dp"
30.	android:layout_height="wrap_content"
31.	android:layout_weight="1"
32.	android:orientation="vertical"
33.	android:paddingStart="10dp">
34.	
35.	<TextView
36.	android:id="@+id/tvTitle"
37.	android:layout_width="match_parent"
38.	android:layout_height="wrap_content"
39.	android:textColor="@color/text_primary"
40.	android:textSize="15sp"
41.	android:textStyle="bold"
42.	android:maxLines="2"
43.	android:ellipsize="end"/>
44.	
45.	<LinearLayout
46.	android:layout_width="match_parent"
47.	android:layout_height="wrap_content"
48.	android:orientation="horizontal"
49.	android:gravity="center_vertical"
50.	android:layout_marginTop="3dp">

51.	
52.	<TextView
53.	android:id="@+id/tvYear"
54.	android:layout_width="wrap_content"
55.	android:layout_height="wrap_content"
56.	android:textColor="@color/text_secondary"
57.	android:textSize="11sp"
58.	android:layout_marginEnd="6dp"/>
59.	
60.	<TextView
61.	android:id="@+id/tvGenre"
62.	android:layout_width="wrap_content"
63.	android:layout_height="wrap_content"
64.	android:layout_weight="1"
65.	android:background="@drawable/badge_background"
66.	android:paddingHorizontal="6dp"
67.	android:paddingVertical="2dp"
68.	android:textColor="@color/accent_color"
69.	android:textSize="10sp"
70.	android:maxLines="1" />
71.	</LinearLayout>
72.	
73.	<LinearLayout
74.	android:layout_width="match_parent"
75.	android:layout_height="wrap_content"
76.	android:orientation="horizontal"
77.	android:layout_marginTop="3dp">
78.	
79.	<TextView

80.	android:id="@+id/tvRating"
81.	android:layout_width="0dp"
82.	android:layout_height="wrap_content"
83.	android:layout_weight="1"
84.	android:textColor="@color/rating_color"
85.	android:textSize="11sp"
86.	android:textStyle="bold"/>
87.	
88.	<TextView
89.	android:id="@+id/tvEpisodes"
90.	android:layout_width="wrap_content"
91.	android:layout_height="wrap_content"
92.	android:textColor="@color/text_secondary"
93.	android:textSize="10sp"/>
94.	</LinearLayout>
95.	
96.	<TextView
97.	android:id="@+id/tvPlot"
98.	android:layout_width="match_parent"
99.	android:layout_height="wrap_content"
100.	android:layout_marginTop="3dp"
101.	android:maxLines="2"
102.	android:ellipsize="end"
103.	android:textColor="@color/text_secondary"
104.	android:textSize="11sp"
105.	android:lineSpacingMultiplier="1.3"/>
106.	
107.	<LinearLayout
108.	android:layout_width="match_parent"

109.	android:layout_height="wrap_content"
110.	android:orientation="horizontal"
111.	android:layout_marginTop="8dp"
112.	android:gravity="start">
113.	
114.	<com.google.android.material.button.MaterialButton
115.	android:id="@+id/btnImdb"
116.	android:layout_width="wrap_content"
117.	android:layout_height="36dp"
118.	android:text="@string/btn_imdb"
119.	android:textSize="10sp"
120.	android:minWidth="0dp"
121.	android:paddingHorizontal="8dp"
122.	app:cornerRadius="16dp"
123.	style="@style/Widget.MaterialComponents.Button.OutlinedButton"
124.	app:strokeColor="@color/accent_color"
125.	android:textColor="@color/accent_color"
126.	android:layout_marginEnd="6dp"/>
127.	
128.	<com.google.android.material.button.MaterialButton
129.	android:id="@+id/btnDetail"
130.	android:layout_width="wrap_content"
131.	android:layout_height="36dp"
132.	android:text="@string/btn_detail"
133.	android:textSize="10sp"
134.	android:minWidth="0dp"
135.	android:paddingHorizontal="8dp"
136.	app:cornerRadius="16dp"

137.	style="@style/Widget.MaterialComponents.Button"
138.	app:backgroundTint="@color/accent_color"
139.	android:layout_marginEnd="6dp"/>
140.	
141.	<com.google.android.material.button.MaterialButton
142.	android:id="@+id/btnFavorite"
143.	style="@style/Widget.MaterialComponents.Button.OutlinedButton"
144.	android:layout_width="36dp"
145.	android:layout_height="36dp"
146.	android:layout_marginEnd="6dp"
147.	android:layout_marginBottom="2dp"
148.	android:layout_marginTop="3dp"
149.	android:minWidth="0dp"
150.	app:cornerRadius="16dp"
151.	app:icon="@drawable/ic_favorite_border"
152.	app:iconGravity="textStart"
153.	app:iconPadding="0dp"
154.	app:iconSize="18dp"
155.	app:iconTint="@color/rating_color"
156.	app:strokeColor="@color/rating_color" />
157.	
158.	</LinearLayout>
159.	</LinearLayout>
160.	</LinearLayout>
161.	
162.	</com.google.android.material.card.MaterialCardView>

Tabel 16.nav_graph.xml

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<navigation
3.	xmlns:android="http://schemas.android.com/apk/res/android"
4.	xmlns:app="http://schemas.android.com/apk/res-auto"
5.	xmlns:tools="http://schemas.android.com/tools"
6.	android:id="@+id/nav_graph"
7.	app:startDestination="@id/homeFragment">
8.	
9.	<fragment
10.	android:id="@+id/homeFragment"
11.	android:name="com.example.animepopular.ui.home.HomeFragment"
12.	android:label="@string/app_name"
13.	tools:layout="@layout/fragment_home">
14.	<action android:id="@+id/action_homeFragment_to_detailFragment" app:destination="@id/detailFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/>
15.	<action android:id="@+id/action_homeFragment_to_languageFragment" app:destination="@id/languageFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/>
16.	<action android:id="@+id/action_homeFragment_to_genreFragment" app:destination="@id/genreFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/>
17.	<action android:id="@+id/action_homeFragment_to_newsFragment" app:destination="@id/newsFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left"

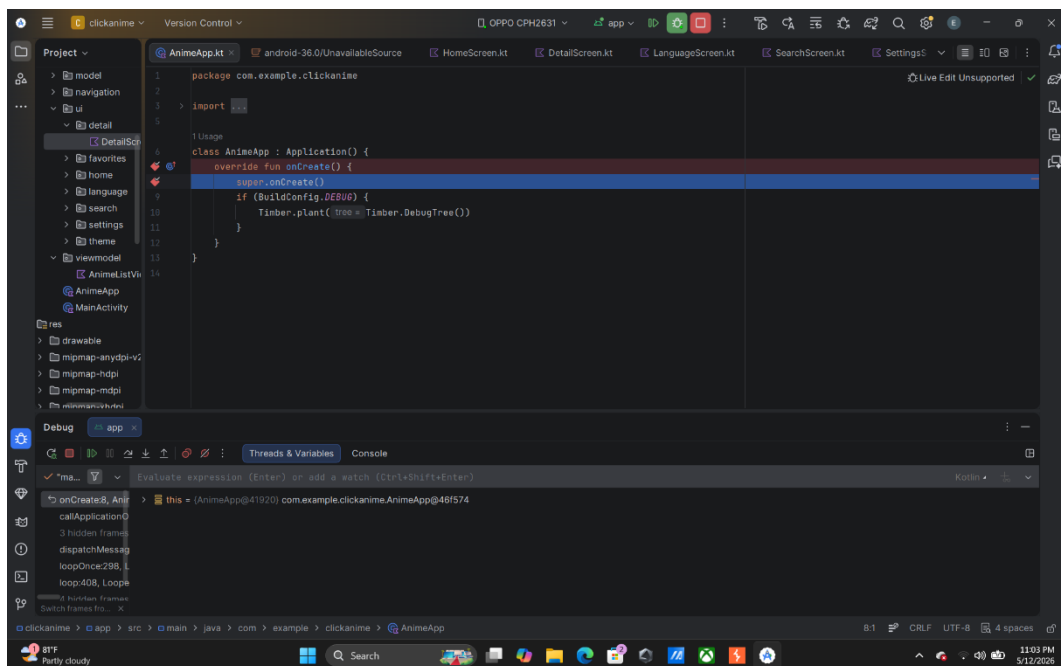
	app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/>
18.	<action android:id="@+id/action_homeFragment_to_scheduleFragment" app:destination="@id/scheduleFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/>
19.	</fragment>
20.	
21.	<fragment
22.	android:id="@+id/detailFragment"
23.	android:name="com.example.animepopular.ui.detail.DetailFragment"
24.	android:label="@string/label_detail"
25.	tools:layout="@layout/fragment_detail">
26.	<argument android:name="anime" app:argType="com.example.animepopular.model.AnimeItem"/>
27.	</fragment>
28.	
29.	<fragment
30.	android:id="@+id/languageFragment"
31.	android:name="com.example.animepopular.ui.language.LanguageFragment"
32.	android:label="@string/label_language"
33.	tools:layout="@layout/fragment_language"/>
34.	
35.	<fragment
36.	android:id="@+id/searchFragment"
37.	android:name="com.example.animepopular.ui.feature.SearchFragment"
38.	android:label="@string/searchanime"
39.	tools:layout="@layout/fragment_search">

40.	<code><action android:id="@+id/action_searchFragment_to_detailFragment" app:destination="@id/detailFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/></code>
41.	<code></fragment></code>
42.	
43.	<code><fragment</code>
44.	<code>android:id="@+id/favoritesFragment"</code>
45.	<code>android:name="com.example.animepopular.ui.feature.FavoritesFragment"</code>
46.	<code>android:label="@string/favorite"</code>
47.	<code>tools:layout="@layout/fragment_favorites"></code>
48.	<code><action android:id="@+id/action_favoritesFragment_to_detailFragment" app:destination="@id/detailFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/></code>
49.	<code></fragment></code>
50.	
51.	<code><fragment</code>
52.	<code>android:id="@+id/topRatedFragment"</code>
53.	<code>android:name="com.example.animepopular.ui.feature.TopRatedFragment"</code>
54.	<code>android:label="@string/toprated"</code>
55.	<code>tools:layout="@layout/fragment_top Rated"></code>
56.	<code><action android:id="@+id/action_topRatedFragment_to_detailFragment" app:destination="@id/detailFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left"</code>

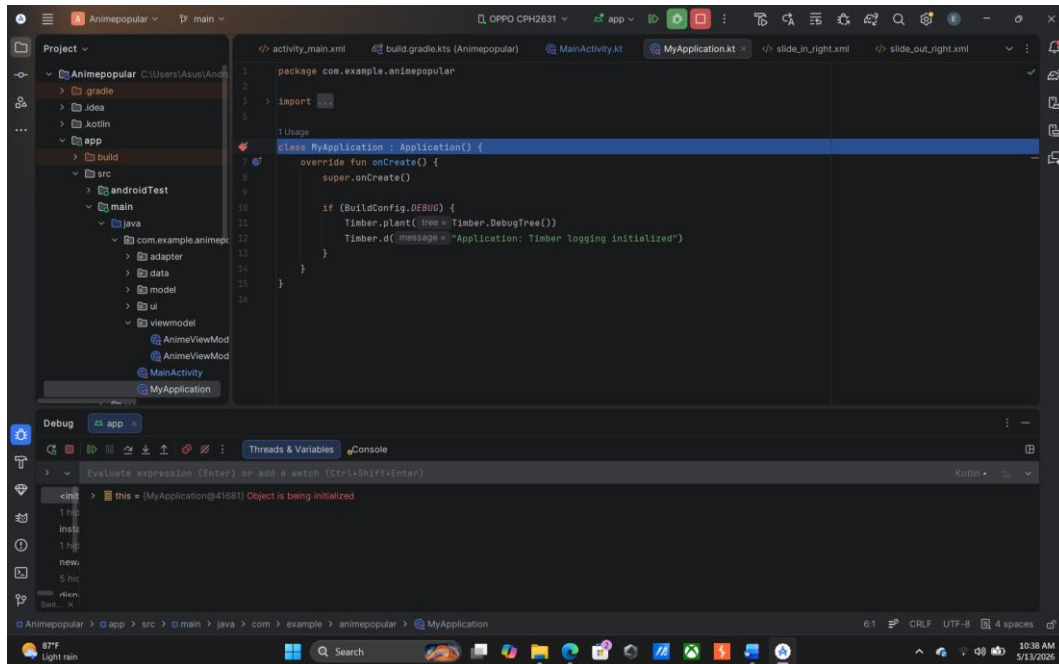
	app:popEnterAnim="@anim/slide_in_left"
	app:popExitAnim="@anim/slide_out_right"/>
57.	</fragment>
58.	
59.	<fragment
60.	android:id="@+id/genreFragment"
61.	android:name="com.example.animepopular.ui.feature.GenreFragment"
62.	android:label="@string/Genre"
63.	tools:layout="@layout/fragment_genre">
64.	<argument android:name="genre" app:argType="string"
	android:defaultValue="Action"/>
65.	<action android:id="@+id/action_genreFragment_to_detailFragment"
	app:destination="@id/detailFragment"
	app:enterAnim="@anim/slide_in_right"
	app:exitAnim="@anim/slide_out_left"
	app:popEnterAnim="@anim/slide_in_left"
	app:popExitAnim="@anim/slide_out_right"/>
66.	</fragment>
67.	
68.	<fragment
69.	android:id="@+id/scheduleFragment"
70.	android:name="com.example.animepopular.ui.feature.ScheduleFragment"
71.	android:label="@string/airing"
72.	tools:layout="@layout/fragment_schedule"/>
73.	
74.	<fragment
75.	android:id="@+id/newsFragment"
76.	android:name="com.example.animepopular.ui.feature.NewsFragment"
77.	android:label="@string/News"
78.	tools:layout="@layout/fragment_news"/>
79.	

80.	<fragment
81.	android:id="@+id/profileFragment"
82.	android:name="com.example.animepopular.ui.about.ProfileFragment"
83.	android:label="@string/profile"
84.	tools:layout="@layout/fragment_profile">
85.	<action
	android:id="@+id/action_profileFragment_to_settingsFragment"
	app:destination="@id/settingsFragment"
	app:enterAnim="@anim/slide_in_right"
	app:exitAnim="@anim/slide_out_left"
	app:popEnterAnim="@anim/slide_in_left"
	app:popExitAnim="@anim/slide_out_right"/>
86.	<action android:id="@+id/action_profileFragment_to_aboutFragment"
	app:destination="@id/aboutFragment"
	app:enterAnim="@anim/slide_in_right"
	app:exitAnim="@anim/slide_out_left"
	app:popEnterAnim="@anim/slide_in_left"
	app:popExitAnim="@anim/slide_out_right"/>
87.	</fragment>
88.	
89.	<fragment
90.	android:id="@+id/settingsFragment"
91.	android:name="com.example.animepopular.ui.about.SettingsFragment"
92.	android:label="@string/settings"
93.	tools:layout="@layout/fragment_settings"/>
94.	
95.	<fragment
96.	android:id="@+id/aboutFragment"
97.	android:name="com.example.animepopular.ui.about.AboutFragment"
98.	android:label="@string/about"
99.	tools:layout="@layout/fragment_about"/>
100.	
101.	</navigation>

A. OUTPUT CODE



Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose



ambar 2. Screenshot Hasil Jawaban Soal 1 XML

B. PEMBAHASAN

MainActivity.kt Compose

MainActivity menginisialisasi state pengaturan bahasa (Locale) melalui fungsi `attachBaseContext`, dan membangun UI aplikasi melalui fungsi `setContent` yang dibungkus oleh tema `AnimeAppTheme`. Aplikasi membuat dan menginisialisasi `AnimeListViewModel` menggunakan `AnimeListViewModelFactory`, lalu meneruskan kontrol navigasi ke `NavGraph`. Aliran ini bersifat satu arah, di mana fungsi pada root meneruskan state navigasi agar aplikasi dapat berpindah antar layar secara reaktif.

AnimeApp.kt Compose

AnimeApp menginisialisasi konfigurasi awal di tingkat global (kelas turunan `Application`) saat aplikasi dijalankan pada metode `onCreate`. Modul ini akan memantau *environment* sistem dan mengaktifkan instance library Timber (`Timber.DebugTree()`) khusus jika aplikasi dijalankan pada mode debug (`BuildConfig.DEBUG`) guna memastikan pengelolaan log tidak tumpang tindih pada versi rilis.

AnimeListViewModel.kt Compose

AnimeListViewModel mengelola state dari daftar karakter anime melalui penggunaan `StateFlow` (seperti `_characters`, `_selectedCharacterId`, dan `_openMalUrl`). Aliran data bersifat satu arah (*unidirectional*); seluruh interaksi pengguna (seperti `onCharacterClicked` atau `onOpenMalClicked`) diproses di dalam `ViewModel` dan dicatat melalui fungsi logging Timber, dan perubahan state ini

kemudian dapat direspons oleh layar UI secara otomatis. Factory pada file yang sama menjamin ViewModel terinisiasi dengan benar sesuai siklus *lifecycle* dengan menerima parameter *string logTag*.

NavGraph.kt Compose

NavGraph mengelola graf navigasi aplikasi secara terpusat menggunakan antarmuka NavHost. Komponen ini merender layar (*Screen*) berdasarkan state *route*, misalnya HomeScreen untuk list menu, atau DetailScreen yang otomatis menarik data karakter yang relevan dari *ViewModel* berdasarkan *characterId* sebelum di-render. Seluruh aksi klik perpindahan layar ditangani lewat parameter fungsi (*lambda*).

SearchScreen.kt Compose

SearchScreen membangun UI untuk layar pencarian memanfaatkan tata letak Scaffold dan fungsi input OutlinedTextField. Layar ini memantau state data dari *ViewModel* menggunakan *collectAsState()*. Jika state teks pada kolom pencarian (*query*) diperbarui, sistem akan langsung menyeleksi dan menyaring iterasi daftar anime berdasarkan tahun atau rating secara *real-time*, sehingga UI otomatis me-refresh data tanpa pengolahan ganda di sisi *View*.

MainActivity.kt XML

MainActivity versi XML menghubungkan data dan logika UI menggunakan objek *ActivityMainBinding* pada fungsi *onCreate*, sekaligus mengatur sinkronisasi bahasa (*Locale*) dengan sistem. Alur interaksi tampilan navigasi (*AppBar* dan *BottomNav*) dikelola secara otomatis menggunakan *NavHostFragment*. UI memantau state *destination* navigasi; di mana sistem akan menyembunyikan *BottomNavigationView* bila pengguna keluar dari menu utama (seperti menuju halaman detail) secara otomatis mengikuti pergantian *route*.

MyApplication.kt XML

MyApplication mewarisi *Application* sebagai entri utama sebelum aktivitas apa pun dijalankan. Ini adalah titik inisialisasi untuk komponen global, yakni memicu pohon utilitas dari Timber apabila *BuildConfig.DEBUG* bernilai *true*. Ini menciptakan sistem logging (*logging tree*) transparan untuk *debugging*.

AnimeAdapter.kt XML

AnimeAdapter membangun list dinamis menggunakan *ListAdapter* yang bergantung pada utilitas asinkron *DiffUtil* agar pergantian iterasi memori jauh lebih responsif dibanding standar *RecyclerView*. UI diikat langsung (*binding*) pada *ItemAnimeBinding*, kemudian event pada berbagai tombol (misal klik IMDb atau penambahan Favorites) dieksekusi dengan mendelegasikannya kembali (*callback invoke*) ke antarmuka *lambda* tanpa harus mengeksekusi logika data secara langsung di dalam Adapter.

AnimeItem.kt XML

`AnimeItem` mengonfigurasi struktur entitas data (*Data Class*) dan diintegrasikan dengan tag `@Parcelable`. Anotasi ini memungkinkan seluruh bundle informasi objek seperti id, judul, episode, gambar, hingga URL dikemas/di-*serialize* untuk dilempar dengan aman secara efisien ke Fragment tujuan tanpa melalui proses *parsing* yang berat.

DetailFragment.kt XML

`DetailFragment` mengatur representasi visual detail konten anime via `FragmentDetailBinding` serta mencomot serpihan argumen (`navArgs()`) dari kelas Navigasi pengirim. UI di Fragment beroperasi tanpa kendali *state data source*, melainkan merespons fungsi-fungsi sederhana seperti perenderan visual `Glide`, logika penambahan *Reviews* pengguna baru, sampai pergantian ikon/warna penanda *Watched* yang dikoordinasikan lewat sinkronisasi dengan *FavoritesRepository*.

AnimeViewModel.kt XML

`AnimeViewModel` menyimpan state utama untuk halaman berbasis XML, seperti data list yang di-*load* (`_animeList`) dan event penekanan detail (`_selectedAnime`, `_detailButtonClicked`) menggunakan perantara `StateFlow`. Fragment tidak menyimpan atau memperbarui list sendiri, melainkan memanggil metode (misalnya `onDetailButtonClicked()`) agar diolah oleh `ViewModel` dan menghasilkan output *Timber logs*. Ini mewujudkan arsitektur yang terisolasi dan *unidirectional*.

AnimeViewModelFactory.kt XML

`AnimeViewModelFactory` mengimplementasikan fitur cetak dasar melalui kelas abstrak `ViewModelProvider.Factory`. Pola ini digunakan untuk membuat spesifikasi instance khusus dari `AnimeViewModel` yang meminta parameter tipe `String` (`filterParam`), karena `ViewModel` secara bawaan di dalam kerangka Android tidak dapat dikonstruksi secara langsung menggunakan parameter.

XML resource code

Rangkaian file `activity_main.xml`, `fragment_detail.xml`, dan `item_anime.xml` membangun kanvas interaksi statis secara terukur. Fragment detail mendefinisikan layout dengan `NestedScrollView` agar elemen padat terbaca, sementara `item_anime` menyematkan visual card per item untuk diisi `Adapter`. Pengaturan rute antarkomponen diatur transparan melalui skema `nav_graph.xml` untuk memudahkan konfigurasi argumen perpindahan.

Debugger

Debugger adalah instrumen bawaan di IDE (seperti `Android Studio`) yang berfungsi untuk menghentikan, menganalisis, dan memeriksa alur berjalannya kode program secara seketika (*real-time*). Fitur ini membantu developer melacak state memori, menangkap proses yang mengarah pada *crash/bug*, serta mengobservasi

value dari properti atau variabel tanpa perlu menebak atau mencetak banyak fungsi *Log/Print*.

Cara Menggunakan Debbuger

1. Tempatkan *Breakpoint* pada salah satu baris kode di margin sebelah kiri editor kode. Saat di-klik, muncul bulatan merah.
2. Jalankan aplikasi tidak dengan ikon *Run* biasa (segitiga hijau), melainkan menggunakan mode **Debug** (ikon berbentuk serangga).
3. Saat operasi di aplikasi menyentuh baris kode yang diberi tanda *breakpoint*, status eksekusi aplikasi Anda akan jeda sementara, dan panel tab menu "*Debug*" akan terbuka di bawah. Anda bisa melihat status memori dan nilai variabel pada jendela "Variables" di waktu tersebut.

Step Into, Step Over, Step Out

Step Into digunakan ketika eksekusi *breakpoint* berada di sebuah baris yang memanggil metode atau fungsi. Perintah ini akan masuk secara mendalam ke dalam kode logika fungsi yang dipanggil tersebut untuk memeriksa isinya baris per baris. **Step Over** digunakan untuk mengarahkan eksekusi melangkah ke baris berikutnya di file atau konteks saat ini tanpa perlu masuk ke dalam implementasi detail fungsi di baris tersebut. **Step Out** digunakan bila masuk ke dalam fungsi yang sangat panjang (akibat *Step Into*) dan ingin langsung melangkah keluar menuju titik asal, membiarkan sisa baris kode di dalam fungsi itu diselesaikan seketika di latar belakang.

TAUTAN GIT

<https://github.com/ERISCHA24/PRAKTIKUMMOBILE.git>